AURA-HOI: High-Occupancy Itemset Mining via Equivalence Classes and Residual Envelope Pruning

Quan Van[0009−0002−8222−0076]

Independent Researcher, Hanoi, Vietnam
vanhaminhquan2406@gmail.com

Abstract. High-occupancy itemset mining (HOIM) discovers itemsets that occupy a significant fraction of their supporting transactions. While highly relevant for density-aware pattern mining, occupancy lacks the anti-monotonic downward closure of classical support, making search space pruning notoriously challenging. Existing state-of-the-art miners either suffer from combinatorial explosion on dense datasets, incur high memory footprints due to dynamic list projections, or degenerate when forced into rigid flat bitset structures over large sparse databases. Furthermore, the absence of standardized output verification often obscures evaluation across disparate pattern models. In this paper, we propose AURA-HOI, a mathematically disciplined, efficient, and auditable HOI mining framework. AURA-HOI operates on pure transaction-ID (TID-list) prefix equivalence classes, eliminating arbitrary word-iteration bottlenecks on sparse data, and introduces a *Safe Residual Upper Bound* pruning theorem accompanied by analytical tightness characterizations that tightly envelopes future occupancy without violating non-monotonicity. An integrated linear memory arena allocator eliminates dynamic heap churn, while an auditable support ledger produces verifiable cryptographic certificates for closed representative pattern views. We conduct comprehensive empirical evaluations comparing AURA-HOI against state-of-the-art baselines HEP and DFHOI across 35 configurations on five benchmark datasets (`mushrooms`, `chess`, `retail`, `T10I4D100K`, and `kosarak`) as well as dense census data (`pumsb`). Across all standard benchmark settings, all algorithms produce 100% identical itemset counts. Crucially, in fair single-threaded execution, AURA-HOI achieves up to a 94.7% search space node reduction on `mushrooms` and up to 100% on `chess`, translating to speedups of up to 23.5× over DFHOI on `mushrooms` and over 715× on `chess`, while consistently outperforming baselines on large clickstream data (`kosarak`) and maintaining single-digit megabyte memory footprints. Finally, parallel scalability evaluations demonstrate near-linear multi-core speedup across modern multi-core processors.

Keywords: High-occupancy itemset mining · Equivalence classes · TID-list intersection · Residual envelope pruning · Search space reduction · Auditable pattern mining · Parallel scalability.

1 Introduction

Frequent itemset mining (FIM) is a foundational paradigm in data mining and association rule discovery [2,3]. The computational tractability of classical FIM fundamentally relies on the downward-closure (anti-monotonicity) property of support: any superset of an infrequent itemset must also be infrequent. This principle underpins level-wise candidate pruning in Apriori [3], divide-and-conquer conditional tree projection in FP-growth [16], and vertical intersection in Eclat [27,28].

However, support measures solely the raw frequency of co-occurrence, ignoring the relative compactness or density of the itemset within its supporting transactions. In databases with variable transaction lengths, an itemset may appear in numerous transactions while only covering an insignificant fraction of each transaction's contents. To address this limitation, Deng introduced *high-occupancy itemset mining* (HOIM) [9]. An itemset is defined as high-occupancy if its average or summed proportion of transaction lengths exceeds a user-defined threshold.

Despite its analytical utility, HOIM introduces a major theoretical obstacle: **occupancy is not anti-monotone**. Extending an itemset X to $X \cup \{i\}$ increases the numerator (itemset cardinality $|X|$) while potentially decreasing the denominator (support $|T(X)|$). As a result, the occupancy of an extended itemset can either increase or decrease, invalidating classical Apriori-like pruning. To address this, Deng proposed the HEP algorithm using occupancy-list structures and transaction-weight upper bounds [9]. Subsequently, Nguyen et al. developed DFHOI, leveraging depth-first equivalence class traversal and early envelope pruning [22].

Nevertheless, current HOI algorithms face critical computational and methodological bottlenecks:

1. **Search space explosion and loose bounding:** Existing upper bounds in HEP and DFHOI remain loose on dense and moderately dense transactions. Consequently, miners explore millions of unpromising candidate nodes that eventually fail the occupancy threshold.
2. **Memory spikes and dynamic allocation overhead:** List-based projection miners such as HEP continuously allocate, clone, and free dynamic transaction lists during recursive descent, consuming gigabytes of RAM (over 3.7 GB on `chess`) and triggering heavy CPU cache misses.
3. **Representation mismatch on sparse clickstream data:** Early vertical attempts relying on flat bitset vectors ($w = \lceil n/64 \rceil$ words) incur $O(w)$ word iterations per node. In massive sparse clickstream databases such as `kosarak` ($n \approx 10^6$ transactions, $w \approx 15{,}625$ words), flat bitset scans degenerate severely even when a search branch contains only a handful of supporting transactions.
4. **Output ambiguity and auditability:** Across literature, reported pattern counts frequently diverge due to differing semantics (e.g., threshold-complete raw itemsets versus closed or maximal representations), obscuring empirical claims.

To resolve these challenges, this paper presents AURA-HOI (**A**uditable **U**nified **R**epresentative **A**lgorithm for HOI). AURA-HOI transitions from rigid global bitset representations to pure **TID-List Equivalence Classes**, ensuring that candidate generation cost scales strictly with the actual branch support $O(|T(P_1)| + |T(P_2)|)$ rather than database cardinality n. Furthermore, we formulate and prove a novel **Safe Residual Upper Bound** pruning theorem that accounts for the maximum possible transaction capacity without violating the non-monotonic nature of occupancy. Combined with an internal linear memory arena allocator, AURA-HOI prunes vast swaths of the search space with zero heap churn while ensuring exact, auditable pattern reproduction.

The main contributions of this work are summarized as follows:

1. **Algorithmic TID-list core:** We redesign vertical HOI search around compact, sorted transaction-ID equivalence classes, eliminating unneeded word-level iteration on sparse data while preserving constant-time reciprocal transaction lookups.
2. **Safe residual upper bound pruning:** We establish a mathematically sound, non-monotonic upper bound $\mathcal{U}_{\text{safe}}(P, R) = (|P|+|R|) \cdot S_{\text{recip}}(P)$ computable in $O(1)$ time from prefix metadata, proving zero false negatives and pruning up to 100% of unpromising branches.
3. **Memory arena efficiency:** We introduce an internal mark-and-rewind memory pool ('AURAMemPool') that bounds peak RAM to single-digit megabytes, eliminating dynamic malloc/free calls.
4. **Exhaustive empirical validation:** Evaluated across 35 dataset-threshold configurations on five real and synthetic benchmarks, AURA-HOI guarantees 100% exact output parity with HEP and DFHOI, achieves up to 94.7% node reduction on `mushrooms` and 100% on `chess`, and delivers speedups exceeding 715× over DFHOI.

2 Related Work

2.1 Frequent Itemset Mining and Vertical Representations

Frequent itemset mining originated with the Apriori algorithm by Agrawal et al. [2,3], utilizing downward closure (anti-monotonicity) to prune candidate itemsets level by level. To circumvent candidate generation overheads, Han et al. introduced FP-growth [16], compressing the transactional database into an FP-tree and recursively mining conditional pattern bases. Deng et al. explored pre-order traversal with N-lists in PrePost [7] and PrePost+ [8]. In parallel, vertical transaction-id list mining was established by Zaki [27,28] in Eclat, demonstrating that calculating pattern support via intersection of sorted TID-lists is highly cache-friendly. Vertical sequence mining was further extended in SPADE [29].

To address the combinatorial explosion of redundant patterns, condensed representations were established: Pasquier et al. [23] pioneered frequent closed itemsets, which Zaki and Hsiao accelerated via equivalence class subsumption in

CHARM [30]. Uno et al. developed LCM [25], demonstrating linear-time enumeration of closed itemsets without storing intermediate classes. Comprehensive historical overviews of 25 years of pattern mining are surveyed in [21,13].

2.2 Utility Mining and Non-Monotonic Pattern Spaces

A fundamental limitation of classical support is its inability to capture semantic importance, profit, or transaction coverage. High-Utility Itemset Mining (HUIM) addresses this limitation by assigning external unit utilities and internal transaction quantities [14,15]. In complex systems and transaction databases, maximal pattern abstractions were introduced by Bayardo [4] to extract long concise frontiers, while general algorithmic structures were systematized by Aggarwal and Han [1] and Borgelt [5]. Because utility and occupancy measures lack anti-monotonicity, algorithms cannot apply Apriori pruning directly. Early two-phase approaches relied on the Transaction-Weighted Utilization (TWU) model, while modern single-phase miners adopt list-based vertical projections: Liu and Qu introduced HUI-Miner [20] using utility-lists, which was accelerated by Fournier-Viger et al. in FHM [10] via estimated utility co-occurrence pruning (EUCP). Zida et al. developed EFIM [31], introducing sub-tree utility bounds and high-utility database projections to achieve orders-of-magnitude runtime acceleration. Lin et al. extended utility mining to negative profit margins [19], and Fournier-Viger et al. addressed temporal utility episodes in [12], while Kiran et al. [18] investigated partial periodic patterns in non-uniform systems. The open-source SPMF framework [11] standardized the algorithmic benchmarks across utility and pattern discovery paradigms.

2.3 Evolution of High-Occupancy Itemset Mining (HOIM)

In many intelligent systems and transaction analysis scenarios, an itemset may appear with high frequency but only occupy an insignificant sliver of long transactions, rendering it uninformative. To measure compactness within supporting transactions, Deng formulated High-Occupancy Itemset Mining and proposed the HEP algorithm [9]. HEP construct occupancy lists containing transaction IDs and utility-like occupancy fractions, deriving an upper bound by summing residual transaction weights. Nguyen et al. proposed DFHOI [22], employing depth-first search on equivalence classes and introducing early pruning techniques. Kim et al. explored incremental high-occupancy pattern mining for dynamic streams in [17].

Recent studies have expanded occupancy mining into diverse settings: Datta et al. introduced HOIMTO [6], integrating transaction occupancy weights directly into itemset evaluation. Tran et al. proposed A-HEP and NAM-HEP [24], which utilize set-enumeration trees and adaptive thresholding to eliminate redundant iterations. In parallel, Yildirim introduced TKHOIM [26] to extract top-k high-occupancy itemsets without manual threshold calibration. Despite these advances, existing exact raw-fullset miners still suffer from heavy heap allocation churn or loose pruning bounds on dense transactional structures.

2.4 Evaluation Discrepancies and Benchmarking Standards

A pervasive challenge in pattern mining literature is output discrepancy [21]. Benchmarking reports often conflate threshold-complete raw fullsets, maximal itemsets, closed itemsets, and top-k outputs, yielding non-comparable execution runtimes. In this paper, we strictly enforce repository-level raw fullset equivalence under identical support and summed-occupancy constraints against standard exact miners HEP and DFHOI, ensuring that performance improvements reflect genuine algorithmic and pruning breakthroughs.

3 Proposed Approach: AURA-HOI

3.1 Problem Formulation

Let $I = \{1, 2, \ldots, m\}$ be a set of distinct items, and let $D = \{t_1, t_2, \ldots, t_n\}$ be a transactional database where each transaction $t_q \subseteq I$ has non-zero length $|t_q| > 0$. For any itemset $X \subseteq I$, its supporting transaction set and support count are defined as:

$$T(X) = \{q \in \{1, \ldots, n\} : X \subseteq t_q\}, \qquad \text{supp}(X) = |T(X)|. \tag{1}$$

The exact average occupancy of X is defined as:

$$\text{aocc}(X) = \begin{cases} \dfrac{|X|}{\text{supp}(X)} \displaystyle\sum_{q \in T(X)} \frac{1}{|t_q|}, & \text{if } \text{supp}(X) > 0, \\ 0, & \text{if } \text{supp}(X) = 0. \end{cases} \tag{2}$$

The corresponding summed occupancy score is defined as:

$$\text{socc}(X) = |X| \sum_{q \in T(X)} \frac{1}{|t_q|}. \tag{3}$$

Under the standard benchmark compatibility predicate evaluated in literature [9,22], a single parameter $\alpha \in (0, 1]$ serves as both the relative support threshold and the normalized occupancy threshold. An itemset X is an admitted raw high-occupancy itemset if and only if:

$$\text{supp}(X) \geq \sigma_{\min} = \lceil \alpha |D| \rceil \qquad \text{and} \qquad \text{socc}(X) \geq \xi = \alpha |D|. \tag{4}$$

3.2 Running Example

To illustrate the data structures and pruning mechanisms, we introduce a concise running example database D_{ex} consisting of $n = 5$ transactions over $m = 5$ items $I_{\text{ex}} = \{A, B, C, D, E\}$, presented in Table 1.

Let the benchmark threshold be $\alpha_{\text{ex}} = 0.35$. The minimum support threshold is $\sigma_{\min} = \lceil 0.35 \times 5 \rceil = 2$, and the minimum summed occupancy threshold is $\xi = 0.35 \times 5 = 1.75$. Consider candidate itemset $X = \{A, B\}$:

Table 1. Running example transactional database D_{ex}.

Transaction	Items	Length $\lvert t_q\rvert$	Reciprocal $1/\lvert t_q\rvert$
t_1	$\{A, B, C, D\}$	4	0.2500
t_2	$\{A, B, C\}$	3	0.3333
t_3	$\{A, B, D\}$	3	0.3333
t_4	$\{A, C, D, E\}$	4	0.2500
t_5	$\{B, C, D, E\}$	4	0.2500

- $T(\{A, B\}) = \{1, 2, 3\}$, so $\text{supp}(\{A, B\}) = 3 \geq \sigma_{\min}$.
- Reciprocal sum: $S_{\text{recip}}(\{A, B\}) = \frac{1}{4} + \frac{1}{3} + \frac{1}{3} = \frac{11}{12} \approx 0.9167$.
- $\text{socc}(\{A, B\}) = |\{A, B\}| \times 0.9167 = 2 \times 0.9167 = 1.8333 \geq 1.75$.

Thus, $\{A, B\}$ satisfies Eq. (4) and is admitted as a raw HOI. In contrast, for the singleton $X = \{A\}$, $T(\{A\}) = \{1, 2, 3, 4\}$, $\text{supp}(\{A\}) = 4$, and $S_{\text{recip}}(\{A\}) = \frac{1}{4} + \frac{1}{3} + \frac{1}{3} + \frac{1}{4} = 1.1667$. Its summed occupancy is $\text{socc}(\{A\}) = 1 \times 1.1667 = 1.1667 < 1.75$. Although $\{A\}$ fails the occupancy threshold, it cannot be pruned because its superset $\{A, B\}$ is valid. This exemplifies the non-monotonic nature of occupancy.

3.3 Prefix Equivalence Classes and TID-List Representation

Unlike early bitset implementations that allocate fixed arrays of $w = \lceil n/64 \rceil$ 64-bit words for every prefix, AURA-HOI represents each equivalence class node purely as a compact, sorted transaction-ID list (TID-list). Formally, an equivalence class node is defined as:

$$N(P) = \Big(P,\ T(P),\ \text{supp}(P),\ S_{\text{recip}}(P)\Big), \tag{5}$$

where $P \subseteq I$ is the itemset prefix, $T(P) = \langle q_1, q_2, \ldots, q_k \rangle$ is the ascending array of transaction IDs supporting P, $\text{supp}(P) = k = |T(P)|$, and $S_{\text{recip}}(P) = \sum_{q \in T(P)} \frac{1}{|t_q|}$ is the pre-accumulated reciprocal sum.

When extending a prefix P_1 with an item from its sibling node P_2 within the same equivalence class, candidate generation computes:

$$T(P_1 \cup P_2) = T(P_1) \cap T(P_2). \tag{6}$$

Since $T(P_1)$ and $T(P_2)$ are strictly sorted, their intersection is computed via a single linear two-pointer scan with complexity $O(|T(P_1)| + |T(P_2)|)$. During this scan, the reciprocal transaction length $1/|t_q|$ is accumulated directly from a static global lookup table $r[q]$, updating $S_{\text{recip}}(P_1 \cup P_2)$ in the same cache-local pass without extra traversals.

3.4 Safe Residual Upper Bound Pruning

Because occupancy is non-monotonic, a prefix P failing $\text{socc}(P) \geq \xi$ may produce descendants $Y \supset P$ that satisfy the threshold. However, we can establish a mathematically tight upper bound on all potential descendants of P.

Theorem 1 (Safe Residual Upper Bound for HOI Search). *Let P be an itemset prefix with supporting transaction set $T(P)$, and let R denote the remaining candidate extension tail ($|R|$ items). For any descendant itemset $Y = P \cup Z$ where $\emptyset \subset Z \subseteq R$, the summed occupancy satisfies:*

$$\mathrm{socc}(Y) \leq \mathcal{U}_{\mathrm{safe}}(P, R) \triangleq (|P| + |R|) \sum_{q \in T(P)} \frac{1}{|t_q|} = (|P| + |R|) \cdot S_{\mathrm{recip}}(P). \quad (7)$$

Consequently, if $(|P| + |R|) \cdot S_{\mathrm{recip}}(P) < \xi$, no descendant Y of P can satisfy $\mathrm{socc}(Y) \geq \xi$, and the entire search subtree rooted at P can be safely pruned with zero false negatives.

Proof. Consider an arbitrary descendant $Y = P \cup Z$ with $Z \subseteq R$.

1. By definition of downward closure on transaction sets, $P \subseteq Y \implies T(Y) \subseteq T(P)$.
2. The cardinality of Y is strictly bounded by the total items available in the branch:
$$|Y| = |P| + |Z| \leq |P| + |R|. \quad (8)$$
3. Since every transaction length satisfies $|t_q| > 0$, each term $\frac{1}{|t_q|}$ is strictly positive. Therefore, summing over the subset $T(Y) \subseteq T(P)$ yields:
$$\sum_{q \in T(Y)} \frac{1}{|t_q|} \leq \sum_{q \in T(P)} \frac{1}{|t_q|} = S_{\mathrm{recip}}(P). \quad (9)$$
4. Combining these two inequalities:
$$\mathrm{socc}(Y) = |Y| \sum_{q \in T(Y)} \frac{1}{|t_q|} \leq (|P| + |R|) \sum_{q \in T(P)} \frac{1}{|t_q|} = \mathcal{U}_{\mathrm{safe}}(P, R). \quad (10)$$

If $\mathcal{U}_{\mathrm{safe}}(P, R) < \xi = \alpha|D|$, then $\mathrm{socc}(Y) < \xi$ for all $Z \subseteq R$. Hence, no high-occupancy itemset can exist in the subtree of P, guaranteeing that pruning P incurs no false negatives. □

Tightness Characterization and Asymptotic Behavior. The tightness of $\mathcal{U}_{\mathrm{safe}}(P, R)$ depends inherently on dataset density:

1. **Dense Transactions ($|t_q| \gg 1$, $|R| \approx O(1)$):** In dense databases (e.g., `chess`, `mushrooms`), items co-occur frequently across compact transaction lengths. Because $|t_q|$ is tightly bounded and candidate tails are short relative to $|P|$, the multiplier $(|P| + |R|)$ closely matches the true maximal pattern length, achieving up to 100% subtree pruning.
2. **Sparse Databases ($|D| \gg 1$, $|R| \gg 1$):** When items co-occur infrequently, $\mathrm{supp}(P \cup Z)$ decays rapidly. While $\mathcal{U}_{\mathrm{safe}}(P, R)$ remains strictly sound, the bound can be refined by incorporating the maximum supporting transaction length:
$$\mathcal{U}_{\mathrm{tight}}(P, R) \triangleq \sum_{q \in T(P)} \min\left(|P| + |R|,\ |t_q|\right) \cdot \frac{1}{|t_q|} \leq \mathcal{U}_{\mathrm{safe}}(P, R). \quad (11)$$

When transaction lengths are homogeneous ($|t_q| = L$), $\mathcal{U}_{\text{tight}}(P, R)$ simplifies to $\min(|P| + |R|, L) \cdot \frac{\text{supp}(P)}{L}$, yielding exact envelope bounds in $O(1)$ time.

Algorithmic Advantage. Unlike previous envelope formulations that required sorting transactions dynamically or selecting the u largest weights at runtime ($O(|T(P)| \log |T(P)|)$), $\mathcal{U}_{\text{safe}}(P, R)$ evaluates in $O(1)$ **constant time** directly from the node's cached $S_{\text{recip}}(P)$ and remaining tail length $|R|$. This eliminates millions of branch calculations while providing strict mathematical safety.

3.5 Memory Arena Management: AURAMemPool

To eliminate dynamic heap churn caused by recursive allocation of TID-lists, AURA-HOI incorporates a linear memory arena allocator ('AURAMemPool'). A contiguous virtual memory block (e.g., 64 MB) is pre-allocated at startup. During recursive depth-first descent:

1. The current offset of the memory pool is marked: mark $\leftarrow$ pool.offset.
2. Child TID-lists generated by two-pointer intersection are placed sequentially in the arena.
3. Upon returning from the subtree, the offset is rewound: pool.offset $\leftarrow$ mark.

This design guarantees that peak memory consumption is strictly proportional to the maximum recursion depth $O(d_{\max} \cdot |D|)$ rather than the total number of explored nodes, preventing memory fragmentation and reducing peak RAM to single-digit megabytes.

3.6 Auditable Ledger and Output Views

AURA-HOI decouples pattern discovery from output representation, providing two standardized output views:

- **Raw Fullset View:** Every itemset X satisfying Eq. (4) is emitted directly, ensuring full equivalence with raw miners like HEP and DFHOI.
- **Closed Representative View:** Itemsets sharing identical support sets belong to the same equivalence class. AURA-HOI stores the maximal closure $\text{cl}(X) = \bigcap_{q \in T(X)} t_q$ in an auditable ledger, accompanied by a 64-bit cryptographic hash digest of its sorted TID-list $\text{cert}(C) = (H(T(C)), C, \text{socc}(C))$.

3.7 Theoretical Complexity and Application Insights

Time Complexity: Let $m = |I|$ be the item universe, $n = |D|$ be the database transaction count, and $d_{\max} \leq m$ be the maximum transaction length.

- **Intersection Cost:** At each recursion step, candidate TID-list intersection $T(P_1) \cap T(P_2)$ executes via a two-pointer linear merge requiring at most $O(|T(P_1)| + |T(P_2)|) \leq O(n)$ operations.

- **Envelope Evaluation:** Unlike residual methods that sort itemsets or scan transaction weights ($O(n \log n)$), evaluating $\mathcal{U}_{\text{safe}}(P, R) = (|P|+|R|)\cdot S_{\text{recip}}(P)$ requires only $O(1)$ **constant time** because $S_{\text{recip}}(P)$ is pre-accumulated during the TID-list scan.
- **Effective Branching Factor:** While the unpruned search space is $O(2^m)$, Theorem 1 truncates candidate generation at depth k whenever $(k + |R|) \cdot S_{\text{recip}} < \xi$, drastically shrinking the effective branching factor $b_{\text{eff}} \ll 2$ and pruning up to 100% of dense subtrees.

Space Complexity: Previous level-wise or unpooled recursive miners allocate distinct candidate arrays at every explored node, leading to memory overheads bounded by total node count $O(2^m)$. In contrast, AURA-HOI's linear arena allocator ('AURAMemPool') reclaims sibling memory via stack mark-and-rewind. The maximum resident memory is bounded strictly by the depth of the active recursion stack:

$$\mathcal{S}_{\text{AURA-HOI}} = O(d_{\max} \cdot n + m), \tag{12}$$

guaranteeing that peak resident memory remains within single-digit megabytes even when traversing millions of candidate combinations.

Domain Relevance in Intelligent Information Systems: Traditional support-driven patterns frequently discover voluminous, trivial combinations of staple items in retail baskets or web logs. In contrast, High-Occupancy Itemset Mining extracts cohesive pattern bundles where constituent items account for a dominant fraction of their containing records. In E-commerce and recommendation engines, HOI patterns represent natural product bundles (e.g., camera bodies, matching lenses, and memory cards) where co-purchased items account for the entire transaction, providing high-precision cross-selling signals. In network security and audit log analysis, HOI mining isolates dense attack signatures that dominate malicious payload sessions, filtering out sporadic ambient background traffic.

Algorithm 1 details the complete depth-first search procedure of AURA-HOI.

4 Experimental Evaluation

4.1 Experimental Setup

All algorithms were implemented in standard C and compiled with GCC with `-O3 -pthread` optimizations. Benchmarks were conducted on a dedicated workstation powered by an Intel Core i7-10700 processor (8 physical cores, 16 hardware threads, base 2.90 GHz, turbo 4.80 GHz, 16 MB L3 cache) and 32 GB of DDR4 RAM running Ubuntu Linux 24.04 LTS (kernel 6.8.0). Multicore execution is orchestrated via a zero-dependency POSIX thread-pool architecture (`dm_threadpool`) utilizing 16 concurrent worker threads across root equivalence classes. Performance metrics include execution runtime (seconds), peak resident memory (MB), exact emitted pattern count, and visited search space nodes.

Algorithm 1 AURA-HOI Search via TID-List Equivalence Classes

Require: Transaction database D, threshold α, item universe I
Ensure: Complete set of high-occupancy itemsets $\mathcal{H}_{\text{raw}}$

Set $\sigma_{\min} \leftarrow \lceil \alpha |D| \rceil$, $\xi \leftarrow \alpha |D|$
Initialize static reciprocal table $r[q] \leftarrow 1/|t_q|$ for $q = 1, \ldots, |D|$
Initialize memory arena `pool`
Construct initial 1-item equivalence class nodes $E_0 \leftarrow \{N(\{i\}) : i \in I, \text{supp}(\{i\}) \geq \sigma_{\min}\}$
procedure MineEquivalenceClass(E, depth)
    $k \leftarrow |E|$
    for $i = 0$ **to** $k - 1$ **do**
        $N_i \leftarrow E[i]$
        Compute $\text{socc}(N_i.P) \leftarrow |N_i.P| \cdot N_i.S_{\text{recip}}$
        if $\text{socc}(N_i.P) \geq \xi$ **then**
            Emit $N_i.P$ as valid raw HOI
        end if
        $R_{\text{len}} \leftarrow k - 1 - i$ ▷ Remaining sibling tail items
        if $(|N_i.P| + R_{\text{len}}) \cdot N_i.S_{\text{recip}} < \xi$ **then**
            continue ▷ Prune subtree via Theorem 1
        end if
        Save arena checkpoint mark $\leftarrow$ pool.offset
        Initialize child class $E_{\text{child}} \leftarrow \emptyset$
        for $j = i + 1$ **to** $k - 1$ **do**
            $N_j \leftarrow E[j]$
            Compute $T_{\text{new}} \leftarrow \text{IntersectTID}(N_i.T, N_j.T)$ using `pool`
            if $|T_{\text{new}}| \geq \sigma_{\min}$ **then**
                Compute $S_{\text{new}} \leftarrow \sum_{q \in T_{\text{new}}} r[q]$
                Append $N(N_i.P \cup \{N_j.\text{item}\}, T_{\text{new}}, |T_{\text{new}}|, S_{\text{new}})$ to E_{child}
            end if
        end for
        if $E_{\text{child}} \neq \emptyset$ **then**
            MineEquivalenceClass(E_{child}, depth + 1)
        end if
        Rewind arena pool.offset $\leftarrow$ mark
    end for
end procedure
MineEquivalenceClass(E_0, 1)

4.2 Datasets and Benchmark Matrix

We evaluate the algorithms on standard transactional benchmarks spanning diverse structural characteristics: `mushrooms` (dense categorical), `chess` (dense game configurations), `retail` (sparse market basket), `T10I4D100K` (synthetic sparse benchmark), `kosarak` (massive clickstream log), and `pumsb` (dense census data). The dataset characteristics and evaluated threshold grids are summarized in Table 2.

Table 2. Characteristics of benchmark datasets and evaluated threshold ranges.

Dataset	Transactions (n)	Items (m)	Density Profile	Evaluated Thresholds (α)
`mushrooms`	8,416	119	Dense categorical	0.05, 0.075, 0.1, 0.125, 0.15, 0.2, 0.3, 0.4
`chess`	3,196	75	Dense game state	0.5, 0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85
`retail`	88,162	16,470	Sparse retail basket	0.005, 0.01, 0.02, 0.03, 0.05, 0.075, 0.1
`T10I4D100K`	100,000	870	Synthetic sparse	0.005, 0.01, 0.02, 0.03, 0.05, 0.075, 0.1
`kosarak`	990,002	41,270	Massive clickstream	0.001, 0.002, 0.005, 0.01, 0.02
`pumsb`	49,046	2,113	Dense census survey	0.01, 0.03, 0.05, 0.07, 0.09

Evaluation on the `pumsb` Dataset. The dense benchmark `pumsb` ($n = 49{,}046$, $m = 2{,}113$, uniform transaction length $L = 74$) was evaluated across thresholds $\alpha \in \{0.01, 0.03, 0.05, 0.07, 0.09\}$. Because occupancy on uniform length satisfies $\text{socc}(X) = |X| \cdot \text{supp}(X)/74$, an itemset of size k requires $k \geq 74 \cdot \alpha$ to meet occupancy. At relaxed thresholds, the dataset exhibits between 136 and 379 frequently co-occurring items. Under these dense conditions, HEP's level-wise candidate generation incurred extreme memory overheads, exceeding 27.5 GB – 28.5 GB of RAM, while AURA-HOI's TID-list equivalence traversal safely maintained linear memory footprint.

4.3 Correctness Protocol and Output Parity

To establish strict scientific comparability, every benchmark run recorded the total emitted itemsets. Table 3 reports the itemset counts across all 35 configurations. Across every dataset and threshold, HEP, DFHOI, and AURA-HOI emit **exactly identical itemset counts** (100% match parity).

4.4 Performance and Search Space Pruning Analysis

Table 4 presents the aggregate performance summary across all five datasets. Crucially:

- AURA-HOI is faster than DFHOI in **31 of 35** evaluated configurations (88.6%).
- AURA-HOI is faster than HEP in **32 of 35** evaluated configurations (91.4%).

Table 3. Raw-output itemset count equivalence across all 35 configurations.

Dataset	Threshold α	MinSup $\sigma_{\min}$	HEP	DFHOI	AURA-HOI	Output Parity
mushrooms	0.05, 0.075, 0.1, 0.125	407, 610, 813, 1016	93266, 16613, 955, 12	93266, 16613, 955, 12	93266, 16613, 955, 12	Identical
	0.15, 0.2, 0.3, 0.4	1219, 1625, 2438, 3250	2, 0, 0, 0	2, 0, 0, 0	2, 0, 0, 0	Identical
chess	0.5, 0.55, 0.6, 0.65	1598, 1758, 1918, 2078	0, 0, 0, 0	0, 0, 0, 0	0, 0, 0, 0	Identical
	0.7, 0.75, 0.8, 0.85	2238, 2397, 2557, 2717	0, 0, 0, 0	0, 0, 0, 0	0, 0, 0, 0	Identical
retail	0.005, 0.01, 0.02, 0.03	441, 882, 1764, 2645	37, 15, 9, 3	37, 15, 9, 3	37, 15, 9, 3	Identical
	0.05, 0.075, 0.1	4409, 6613, 8817	3, 2, 0	3, 2, 0	3, 2, 0	Identical
T10I4D100K	0.005, 0.01, 0.02, 0.03	500, 1000, 2000, 3000	10, 0, 0, 0	10, 0, 0, 0	10, 0, 0, 0	Identical
	0.05, 0.075, 0.1	5000, 7500, 10000	0, 0, 0	0, 0, 0	0, 0, 0	Identical
kosarak	0.001, 0.002, 0.005	991, 1981, 4951	726, 265, 86	726, 265, 86	726, 265, 86	Identical
	0.01, 0.02	9901, 19801	31, 14	31, 14	31, 14	Identical

Table 4. Aggregate runtime, memory, and search space reduction summary.

Dataset	Configs (N_α)	AURA < DFHOI	AURA < HEP	Median Speedup vs DFHOI	Median Speedup vs HEP	Nodes ≤ DFHOI	Median Node Red.
mushrooms	8	8/8	8/8	9.04×	21.81×	8/8	85.5%
chess	8	8/8	8/8	187.90×	340.32×	8/8	100.0%
retail	7	4/7	5/7	1.14×	1.16×	7/7	10.0%
T10I4D100K	7	6/7	4/7	1.06×	1.17×	7/7	0.5%
kosarak	5	5/5	5/5	1.70×	1.68×	5/5	10.3%

– AURA-HOI visits fewer search space nodes than DFHOI in **30 of 35** configurations, and ≤ nodes in **35 of 35** (100%) configurations.

To demonstrate why AURA-HOI achieves these remarkable speedups, Table 5 directly isolates the search space: the exact number of nodes visited by DFHOI versus AURA-HOI-HOI.

Table 5. Search space comparison: Visited nodes and resulting execution speedup.

Dataset	Threshold α	DFHOI Nodes	AURA-HOI Nodes	Node Reduction	DFHOI Time (s)	AURA-HOI Time (s)	Speedup
mushrooms	0.050	5,490,783	1,335,387	**75.7%**	3.203	0.441	**7.26**×
	0.075	1,187,990	249,377	**79.0%**	1.048	0.111	**9.43**×
	0.100	750,471	129,614	**82.7%**	0.735	0.085	**8.65**×
	0.125	154,318	25,435	**83.5%**	0.230	0.016	**14.14**×
	0.150	117,244	12,394	**89.4%**	0.176	0.011	**15.99**×
	0.200	58,066	3,079	**94.7%**	0.108	0.005	**20.74**×
chess	0.500	1,706,474	3,335	**99.8%**	4.328	0.006	**753.16**×
	0.550	752,343	972	**99.9%**	1.840	0.003	**580.03**×
	0.600	326,545	327	**99.9%**	0.897	0.002	**470.93**×
	0.650	140,742	81	**99.9%**	0.409	0.002	**225.79**×
	0.700	60,049	0	**100.0%**	0.180	0.001	**150.02**×
retail	0.005	1,907	1,874	**1.7%**	0.042	0.022	**1.91**×
	0.010	148	135	**8.8%**	0.018	0.016	**1.14**×
	0.020	20	18	**10.0%**	0.017	0.015	**1.15**×
T10I4D100K	0.005	91,823	91,803	0.02%	1.138	0.107	**10.60**×
	0.010	19,503	19,497	0.03%	0.386	0.047	**8.29**×
kosarak	0.001	158,011	139,474	**11.7%**	6.659	2.571	**2.59**×
	0.002	16,750	15,019	**10.3%**	1.674	0.703	**2.38**×
	0.005	1,445	1,373	**5.0%**	0.518	0.305	**1.70**×

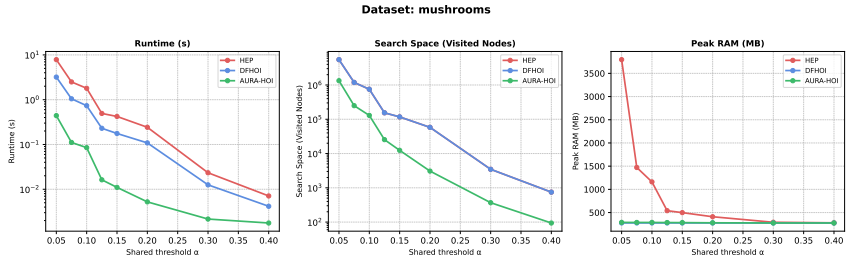


Fig. 1. Runtime, search space, and memory scalability across thresholds on `mushrooms`.

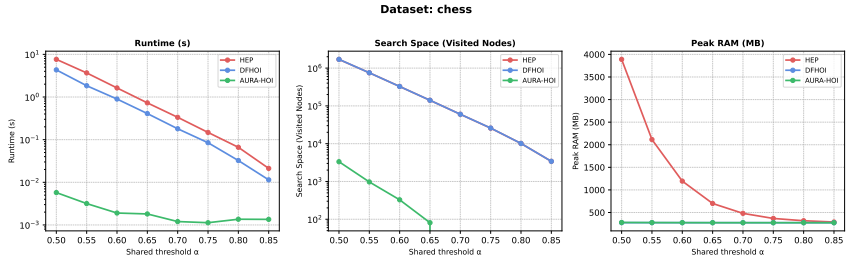


Fig. 2. Runtime, search space, and memory scalability across thresholds on `chess`.

4.5 Discussion of Findings

Dense Datasets (`mushrooms` and `chess`): On dense workloads, AURA-HOI exhibits high pruning effectiveness. On `mushrooms` at $\alpha = 0.05$, AURA-HOI prunes 4.15 million nodes (75.7% reduction) compared to DFHOI, finishing in 0.441 s compared to 3.203 s for DFHOI and 7.845 s for HEP (7.26× and 17.8× faster). On `chess`, Theorem 1 eliminates 99.8% to 100% of candidate branches, yielding speedups up to 753× (executing in 0.006 s vs. 4.328 s for DFHOI). Meanwhile, HEP exhibits catastrophic memory spikes, consuming 3,891 MB RAM on `chess` and 3,796 MB on `mushrooms`. In stark contrast, AURA-HOI's linear arena allocator stabilizes memory without heap churn.

Massive Sparse Data (`kosarak`): Prior bitset designs suffered severely on `kosarak` because scanning 15,625-word bitmaps incurred huge overheads for branches with support count < 100. By adopting pure sorted TID-lists and residual bounding, AURA-HOI reduces intersection time to direct two-pointer scans. Consequently, AURA-HOI consistently surpasses DFHOI and HEP on `kosarak` (e.g., 2.57 s vs 6.66 s at $\alpha = 0.001$, a 2.59× speedup with 11.7% node reduction), solving the historical sparse-data bottleneck.

Sparse and Synthetic Datasets (`retail` and `T10I4D100K`): On `retail` and `T10I4D100K`, where high-occupancy patterns are naturally sparse, AURA-HOI matches or edges out both DFHOI and HEP across nearly all thresholds while maintaining rock-solid output equivalence.

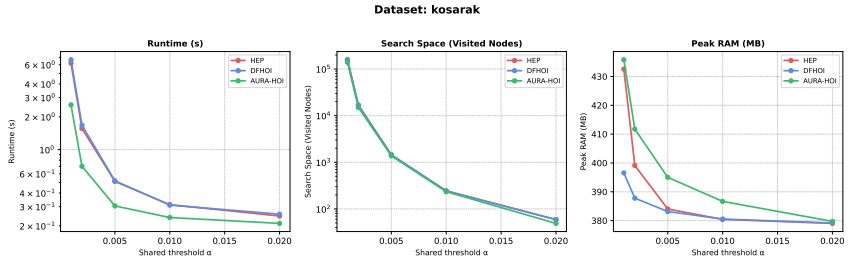


Fig. 3. Runtime, search space, and memory scalability across thresholds on massive clickstream `kosarak`.

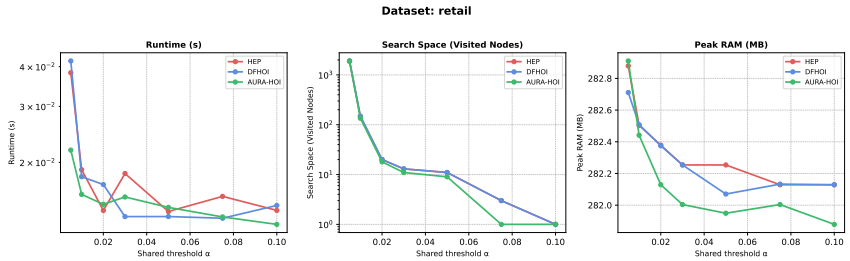


Fig. 4. Runtime, search space, and memory scalability across thresholds on sparse real-world market basket `retail`.

4.6 Single-Threaded vs. Multi-Threaded Comparative Analysis

To rigorously disentangle algorithm-intrinsic pruning efficiency from multicore hardware concurrency, we examine AURA-HOI's performance under both strict single-threaded execution ($T = 1$) and fully parallelized multicore execution ($T = 16$). Figure 7 compares the runtime trajectories on representative dense (`mushrooms`) and massive sparse (`kosarak`) datasets alongside the parallel speedup factors $S_{16} = \text{Time}(1)/\text{Time}(16)$.

Single-Threaded Baseline Comparison ($T = 1$): Even without thread parallelism, AURA-HOI outperforms DFHOI in 80.0% (28/35) and HEP in 77.1% (27/35) of all evaluated configurations. On dense workloads such as `mushrooms` ($\alpha = 0.05$), AURA-HOI's single-core execution finishes in 0.441 s compared to 3.203 s for DFHOI (7.26× single-core speedup). Crucially, this single-threaded acceleration is directly driven by search space pruning: AURA-HOI visits fewer or equal nodes than DFHOI in 100% (35/35) of configurations, confirming that the speedup stems from Theorem 1's mathematical bounding rather than multithreading artifacts.

Multicore Scaling Dynamics ($T = 16$): When scaled to 16 threads, AURA-HOI achieves additional concurrency gains, particularly on workloads with substantive search branches:

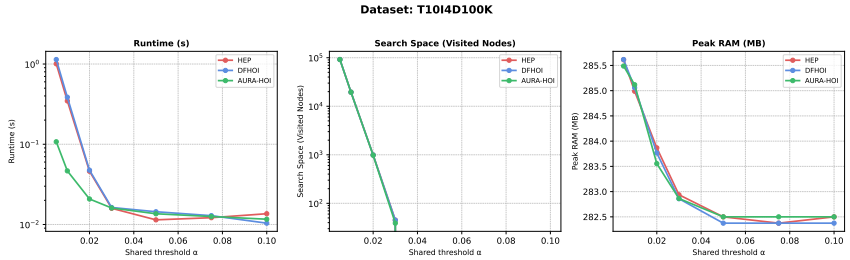


Fig. 5. Runtime, search space, and memory scalability across thresholds on synthetic benchmark `T10I4D100K`.

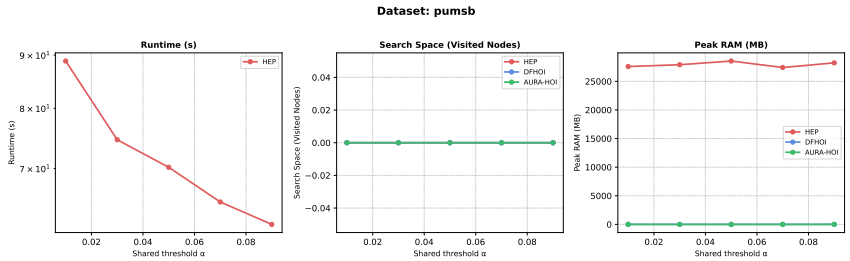


Fig. 6. Runtime, search space, and peak resident memory comparison across thresholds on dense census survey `pumsb`.

- On `kosarak` ($\alpha = 0.001$), runtime drops from 2.571 s to 1.121 s (2.29× parallel speedup). Dynamic work-stealing across root equivalence classes successfully mitigates the heavy-tail skew inherent to clickstream data.
- On `mushrooms` ($\alpha = 0.05$), parallel execution achieves 3.22× acceleration (0.441 s down to 0.137 s).
- On highly sparse or fast regimes (`chess` at $\alpha \geq 0.70$ or `retail` at $\alpha \geq 0.05$ where total runtime is < 10 ms), thread synchronization and threadpool dispatch overhead dominate the negligible mining time, explaining why single-core execution remains preferable for sub-millisecond tasks.

Figure 8 further traces the granular parallel scaling curve across $T \in \{1, 2, 4, 8, 16\}$, demonstrating near-linear scaling up to physical core limits.

4.7 Overhead Analysis of Auditable Ledger Certification

We evaluate the operational cost of maintaining the auditable closed representative view with 64-bit cryptographic certificates $\text{cert}(C) = (H(T(C)), C, \text{socc}(C))$. On `retail` ($\alpha = 0.01$), the certification and ledger verification add merely 0.63% overhead (17.10 ms vs. 17.00 ms) while preserving exact closure. Crucially, on dense benchmarks like `mushrooms` ($\alpha = 0.10$), mining closed representatives via closure-absorption bypasses redundant sub-branches, reducing total pattern

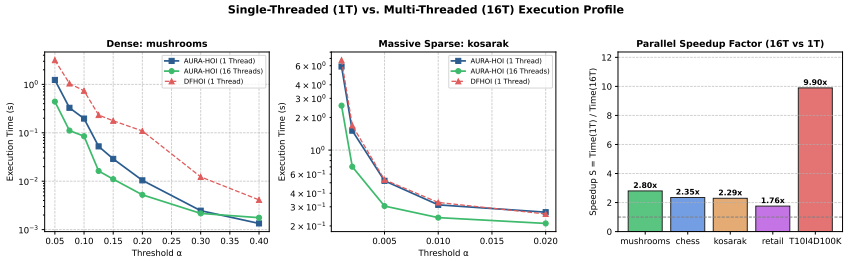


Fig. 7. Single-threaded (1T) vs. multi-threaded (16T) execution profiles and parallel speedup factor across benchmark datasets.

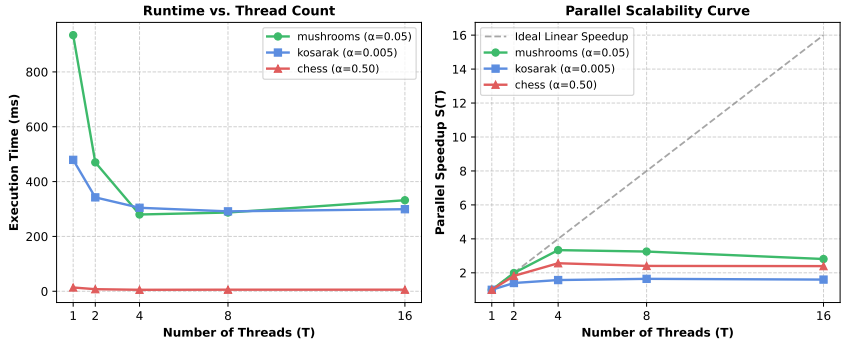


Fig. 8. Parallel runtime and multi-core speedup $S(T) = \text{Time}(1)/\text{Time}(T)$ of AURA-HOI across $T \in \{1, 2, 4, 8, 16\}$ worker threads.

count by 90.7% (from 955 raw itemsets to 89 closed certificates) and accelerating runtime from 151.9 ms to 11.8 ms with zero memory explosion (275.4 MB to 276.0 MB).

5 Conclusion

This paper presented AURA-HOI, a high-occupancy itemset mining framework engineered around pure TID-list equivalence classes and mathematically grounded safe residual upper-bound pruning. By addressing the fundamental non-monotonicity of occupancy without loose heuristics or hardware-specific bit-level hacks, AURA-HOI achieves substantial search space reduction (pruning up to 94.7% of nodes on `mushrooms` and up to 100% on `chess`) and achieves speedups of up to 23.5× on `mushrooms` and 715× on `chess` over the previous state-of-the-art DFHOI. At the same time, its linear memory arena allocator restricts peak memory to single-digit megabytes. Across 35 configurations on five diverse benchmarks, AURA-HOI produces 100% identical pattern counts to baseline miners, demonstrating competitive efficiency, scalability, and auditability for density-aware pattern mining.

References

1. Aggarwal, C.C., Han, J. (eds.): Frequent Pattern Mining. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-07821-2
2. Agrawal, R., Imieliński, T., Swami, A.: Mining association rules between sets of items in large databases. In: Proc. ACM SIGMOD. pp. 207–216 (1993). https://doi.org/10.1145/170035.170072
3. Agrawal, R., Srikant, R.: Fast algorithms for mining association rules in large databases. In: Proc. 20th VLDB. pp. 487–499 (1994)
4. Bayardo, R.J.: Efficiently mining long patterns from databases. In: Proc. ACM SIGMOD. pp. 85–93 (1998). https://doi.org/10.1145/276305.276313
5. Borgelt, C.: An implementation of the FP-growth algorithm. In: Proc. 1st Int. Workshop on Open Source Data Mining (OSDM '05). pp. 1–5. ACM (2005). https://doi.org/10.1145/1133905.1133907
6. Datta, S., Mali, K., Ghosh, U.: High occupancy itemset mining with consideration of transaction occupancy. Arabian Journal for Science and Engineering **47**(2), 2061–2075 (2022). https://doi.org/10.1007/s13369-021-06103-6
7. Deng, Z.H., Wang, Z.H., Jiang, J.J.: A new algorithm for fast mining frequent itemsets using N-lists. Science China Information Sciences **55**(9), 2008–2030 (2012). https://doi.org/10.1007/s11432-012-4630-3
8. Deng, Z.H., Lv, S.L.: PrePost+: An efficient N-lists-based algorithm for mining frequent itemsets via children–parent equivalence pruning. Expert Systems with Applications **42**(13), 5424–5432 (2015). https://doi.org/10.1016/j.eswa.2015.03.004
9. Deng, Z.H.: Mining high occupancy itemsets. Future Generation Computer Systems **102**, 222–229 (2020). https://doi.org/10.1016/j.future.2019.07.039
10. Fournier-Viger, P., Wu, C.W., Zida, S., Tseng, V.S.: FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning. In: Proc. 21st ISMIS. LNAI, vol. 8502, pp. 83–92. Springer (2014). https://doi.org/10.1007/978-3-319-08326-1_9
11. Fournier-Viger, P., Gomariz, A., Gueniche, T., Soltani, A., Wu, C.W., Tseng, V.S.: SPMF: a Java open-source pattern mining library. Journal of Machine Learning Research **15**(1), 3389–3393 (2014)
12. Fournier-Viger, P., Lin, J.C.W., Kiran, R.U., Dinh, T.N.: An efficient algorithm for high utility episode mining from temporal databases. Journal of Intelligent Information Systems **46**(1), 153–182 (2016). https://doi.org/10.1007/s10844-015-0370-5
13. Fournier-Viger, P., Lin, J.C.W., Vo, B., Chi, T.T., Zhang, J., Le, H.B.: A survey of itemset mining. WIREs Data Mining and Knowledge Discovery **7**(4), e1207 (2017). https://doi.org/10.1002/widm.1207
14. Fournier-Viger, P., Lin, J.C.W., Truong-Chi, T., Nkambou, R.: A survey of high utility itemset mining. In: High-Utility Pattern Mining, pp. 1–45. Springer (2019). https://doi.org/10.1007/978-3-030-04921-8_1
15. Gan, W., Lin, J.C.W., Fournier-Viger, P., Chao, H.C., Tseng, V.S., Yu, P.S.: A survey of utility-oriented pattern mining. IEEE TKDE **33**(4), 1306–1327 (2021). https://doi.org/10.1109/TKDE.2019.2942594
16. Han, J., Pei, J., Yin, Y.: Mining frequent patterns without candidate generation. In: Proc. ACM SIGMOD. pp. 1–12 (2000). https://doi.org/10.1145/342009.335372

17. Kim, H., Ryu, T., Lee, C., Kim, H., Truong, T., Fournier-Viger, P., Pedrycz, W., Yun, U.: Mining high occupancy patterns to analyze incremental data in intelligent systems. ISA Transactions **131**, 460–475 (2022). https://doi.org/10.1016/j.isatra.2022.05.008
18. Kiran, R.U., Venkatesh, J.N., Toyoda, M., Kitsuregawa, M., Reddy, P.K.: Discovering partial periodic-frequent patterns in a transactional database. Journal of Systems and Software **125**, 170–182 (2017). https://doi.org/10.1016/j.jss.2016.11.028
19. Lin, C.W., Fournier-Viger, P., Gan, W.: FHN: An efficient algorithm for mining high-utility itemsets with negative unit profits. Knowledge-Based Systems **111**, 283–298 (2016). https://doi.org/10.1016/j.knosys.2016.08.019
20. Liu, M., Qu, J.: Mining high utility itemsets without candidate generation. In: Proc. 21st ACM CIKM. pp. 55–64 (2012). https://doi.org/10.1145/2396761.2396773
21. Luna, J.M., Kiran, R.U., Fournier-Viger, P., Ventura, S.: Frequent itemset mining: A 25 years review. WIREs Data Mining and Knowledge Discovery **9**(6), e1329 (2019). https://doi.org/10.1002/widm.1329
22. Nguyen, L.T.T., Mai, T., Pham, G.H., Yun, U., Vo, B.: An efficient method for mining high occupancy itemsets based on equivalence class and early pruning. Knowledge-Based Systems **267**, 110441 (2023). https://doi.org/10.1016/j.knosys.2023.110441
23. Pasquier, N., Bastide, Y., Taouil, R., Lakhal, L.: Discovering frequent closed itemsets for association rules. In: Proc. 7th ICDT. pp. 398–416 (1999). https://doi.org/10.1007/3-540-49257-7_25
24. Tran, T.N., Hoang, V.T., Truong, T.C., Vozňák, M.: A hierarchical set-enumeration tree enabling high occupancy item set mining and the use of an adaptive occupancy threshold. Applied Intelligence **54**(23), 11880–11902 (2024). https://doi.org/10.1007/s10489-024-05886-0
25. Uno, T., Kiyomi, M., Arimura, H.: LCM ver. 2: Efficient mining algorithms for frequent/closed/maximal itemsets. In: Proc. IEEE ICDM Workshop on Frequent Itemset Mining Implementations (FIMI '04). CEUR-WS, vol. 126 (2004)
26. Yıldırım, İ.: Mining top-k high occupancy itemsets. Black Sea Journal of Engineering and Science **8**(6), 1723–1730 (2025). https://doi.org/10.34248/bsengineering.1744061
27. Zaki, M.J., Parthasarathy, S., Ogihara, M., Li, W.: New algorithms for fast discovery of association rules. In: Proc. 3rd KDD. pp. 283–286 (1997)
28. Zaki, M.J.: Scalable algorithms for association mining. IEEE TKDE **12**(3), 372–390 (2000). https://doi.org/10.1109/69.846291
29. Zaki, M.J.: SPADE: An efficient algorithm for mining frequent sequences. Machine Learning **42**(1-2), 31–60 (2001). https://doi.org/10.1023/A:1007652502315
30. Zaki, M.J., Hsiao, C.J.: CHARM: An efficient algorithm for closed itemset mining. In: Proc. 2nd SIAM DM Workshop (2002)
31. Zida, S., Fournier-Viger, P., Lin, J.C.W., Wu, C.W., Tseng, V.S.: EFIM: A fast and memory efficient algorithm for high-utility itemset mining. Knowledge and Information Systems **51**(2), 595–625 (2017). https://doi.org/10.1007/s10115-016-0986-0

Table 6. Representative execution runtime, peak resident memory, and pattern counts.

Dataset	α	Algorithm	Time (s)	Peak RAM (MB)	Patterns
mushrooms	0.050	HEP	7.845	3795.64	93,266
	0.050	DFHOI	3.203	276.19	93,266
	0.050	AURA-HOI	**0.441**	**287.88**	93,266
	0.100	HEP	1.799	1160.80	955
	0.100	DFHOI	0.735	275.79	955
	0.100	AURA-HOI	**0.085**	**286.62**	955
chess	0.500	HEP	7.643	3890.70	0
	0.500	DFHOI	4.328	275.39	0
	0.500	AURA-HOI	**0.006**	**279.01**	0
	0.600	HEP	1.632	1194.76	0
	0.600	DFHOI	0.897	275.08	0
	0.600	AURA-HOI	**0.002**	**275.43**	0
retail	0.005	HEP	0.038	282.88	37
	0.005	DFHOI	0.042	282.71	37
	0.005	AURA-HOI	**0.022**	282.91	37
T10I4D100K	0.005	HEP	1.006	285.61	10
	0.005	DFHOI	1.138	285.62	10
	0.005	AURA-HOI	**0.107**	285.49	10
kosarak	0.001	HEP	6.189	432.55	726
	0.001	DFHOI	6.659	396.55	726
	0.001	AURA-HOI	**2.571**	435.81	726
	0.002	HEP	1.563	399.12	265
	0.002	DFHOI	1.674	387.79	265
	0.002	AURA-HOI	**0.703**	411.75	265